Community Detection in Bacterial and Viral Networks

Jonas Ziegler

February 28, 2026

Abstract

This paper investigates community structure in bacterial and viral protein similarity networks using modularity-based community detection methods. We compare several algorithms and perform a sensitivity analysis on the resolution parameter using Adjusted Rand Index and Adjusted Mutual Information.

1 Introduction

1.1 Motivation

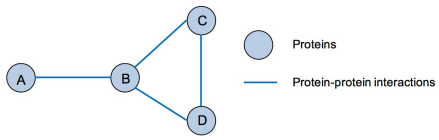
1.1.1 Why statistical network analysis is important

We live in a highly interconnected world where many physical, social, technological and biological systems consist of agents or entities interacting with each other. Examples include a virus being transmitted over social contact networks, global trade between countries, and the human brain. Any such system can be represented as a network by denoting the entities as nodes and the interactions between them as edges. This makes networks an important type of data spanning a remarkable variety of complex systems. It is therefore very important to have mathematically rigorous and practically useful methods for statistical analysis of networks.

1.1.2 Biological Networks

Biological systems are often represented as networks. Essentially, every biological entity has interactions with oder biological entitites, from th emolecular to the ecosystem level, providing us with the oppurtunity to model biology using many different types of netowrks such as ecological, neurogical, metabolic and protein-protein networks.

A protein-protein interaction network (PPI network), also known as the interactome, is a type of biological network where a node represents a protein and an edge represents the physical interaction between each pair of proteins.



«««< HEAD

Figure 1: Protein–protein interaction network. Adapted fromEuropean Bioinformatics Institute (EMBL-EBI) 2024.

=======

Figure 2: Protein–protein interaction network. Adapted from European Bioinformatics Institute (EMBL-EBI) 2024.

»»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872

1.1.3 Why Community Detection

M. E. J. Newman (2006) argues that detecting and characterizing community structure is one of the fundamental problems in the study of networked systems, and notes that many networks of scientific interest — including biological ones — ««««< HEAD naturally divide into communities. Detecting these communities can help us understand ======= naturally divide into communities. Detecting these communitys can help us understand »»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872 the structure and function of the network, as well as the underlying system it represents. For example, in the context of PPI(protein-protein interaction) networks, communities can help identify unknown protein complexes.

1.2 Definition of a Network

1.2.1 Graph as a mathematical object

We call a collection of connected objects a network. In the field of mathematics, a network can also be called a graph. The mathematical theory behind networks is known as graph theory. A graph is a mathematical object that consists of a set of objects (called vetices or nodes) that are connected by links (called edges).

1.2.2 Adjacency list

An adjacency list is a graph representation method that uses an array of lists to store connections between verticec. Each array index represents a vertex, and its corresponding list contains all neighboring nodes, making it highly efficient for sparse graphs($G = (V, E)$, where the number of edges is much less than the maximum possible number of edges). The main operation performed by the adjacency list data structure is to report a list of neighboprs of a given vertex.

1.2.3 Adjacency matrix

The main alternative to the adjacency list is the adjacency matrix, a matrix whose rows and columns are indexed by vertices and the cell (i, j) contains 1 if there is an edge between vertices i and j, or 0 if no such edge exists. The diagonal elements of the matrix are all 0, since edges from a vertex to itself are not allowed in simple graphs.

1.2.4 Directed vs. undirected networks

Undirected and directed graohs are fundamental concepts in grap theory. An undirceted graph is a graph where the edges do not have a specific direction and it is bidirectional in nature meaning it does not have a parent-child relation concept. In contrast, a directed graph is a graph where the edges have a specific direction, indicating a one-way relationship between the vertices.

An important difference between directed and undirected graphs is that in a undirected graph, relationships are symmetric, meaning if vertex A is connected to vertex B, then vertex B is also connected to vertex A. In a directed graph, relationships are asymmetric, meaning if vertex A is connected to vertex B, it does not necessarily imply that vertex B is connected to vertex A. This Characteristic is well represented in the adjacency matrix as you can see in the figure below.

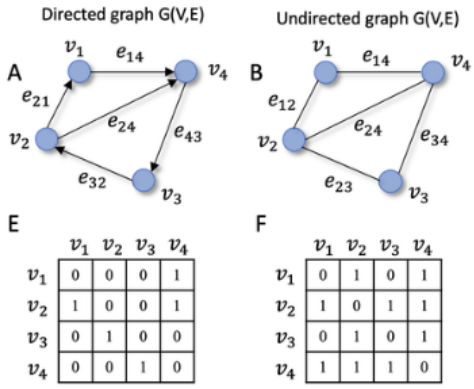


Figure 3: Directed vs. undirected graph. Adapted from ResearchGate 2020.

1.2.5 Weighted undirected network

Because in this paper we will be working with weighted undirected networks, we will now define what a weighted undirected network is. A weighted undirected network is a graph where each edge has a weight associated with it, representing the strength or importance of the connection between two vertices. In such networks, the adjacency matrix is not binary but contains numerical weights for each edge. Mathematically, a weighted undirected network can be represented as $G = (V, E, W)$, where V is the set of vertices, E is the set of edges, and W is a function that assigns a weight to each edge in E.

1.3 Network Analysis

1.3.1 Descriptive statistics

Hevey (2018) states that Network analysis provides the capacity to estimate complex patterns of relationships and the network structure can be analysed to reveal core features the network. A variety of basic descriptive metrics can be used to characterize network topology, including number of nodes and edges, network density, degree distribution and centrality measures. These measures provide an initial understanding of connectivity patterns and structural organization.

1.3.2 Community Detection and Community Comparison

In network science, a community is a group of nodes that are more densely connected to each other than to the rest of the network. Detecting communities can help us understand the functional organization of a system. For example, in a PPI network, groups of proteins that interact often perform the same function in a cell. Community detection methods such as Modularity, Louvain and Leiden algorithms can be used to identify these community structures. For this paper, attention is given to similarity measures for community comparison such as Adjusted Rand Index (ARI) and Adjusted Mutual Information (AMI), which quantify the agreement between different community structures. All these concepts will be explained in more detail in the following sections.

2 Network Summary Statistics

2.1 Basic Metrics

2.1.1 Number of nodes and edges

The number of nodes and edges are fundamental metrics that provide a basic understanding of the size and complexity of a network. The number of nodes, denoted as $|V|$, represents the total count of distinct entities in the network, while the number of edges, denoted as $|E|$, represents the total count of connections between those entities.

2.1.2 Density

Network density is a measure of how many edges are in the network compared to the maximum possible number of edges. THe metric is defined as the ratio of observed edges to the total possible connections, ranging from 0 to 1, where 0 indicates a completely disconnected network and 1 indicates a fully connected network. For an undirected network, the density can be calculated using the formula: «««< HEAD

$$\text{Density} = \frac{|E|}{\binom{|V|}{2}} = \frac{2|E|}{|V|(|V|-1)}$$

=======

$$\text{Density} = \frac{|E|}{\binom{|V|}{2}} = \frac{2|E|}{|V|(|V|-1)}$$

»»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872

2.1.3 Average degree

The average degree of a netork is a network metric representing the mean number of connections per node. The degree of a node refers to the number of edges connected to that node. These edges are called incident edges, meaning edges that are directly connected to the node in question. In an undirected network, the degree simply counts all incident edges, while in a directed network, a distinction is made between in-degree and out-degree. For an undirected network, it can be expressed mathematically as: «««< HEAD

$$\text{Average Degree} = \frac{2|E|}{|V|}$$

For an directed network, it can be expressed mathematically as:

$$\text{Average In-Degree} = \text{Average Out-Degree} = \frac{|E|}{|V|}$$

=======

$$\text{Average Degree} = \frac{2|E|}{|V|}$$

For an directed network, it can be expressed mathematically as:

$$\text{Average In-Degree} = \text{Average Out-Degree} = \frac{|E|}{|V|}$$

»»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872 Average In-Degree means the average number of incoming edges per node, while Average Out-Degree means the average number of outgoing edges per node. They are equal in a directed network because each edge contributes to both an in-degree and an out-degree.

2.2 Degree Distribution

2.2.1 Definition of degree distribution

The degree distribution of a networ describes the probability distribution of the degrees of nodes in a network. It specifies the probability P(k) that a randomy selected node has exactly k connections (or degree k). The distribution provides insighjt into the structural organization of a network and allows the differentiation between homogeneous and heterogeneous networks.

2.2.2 Random graph and scale-free graph

Random graphs and scale-free graphs are two fundamental network models that differs significantly in their degree distributions. A random graph, often modeled using the Erdős–Rényi framework, is construced by connecting nodes randomly with a fixed probability p. In such networks, edges are formed independently of each other. As a result, most nodes tend to have a similar number of connections, leading to a relatively homogeneous structure. Highly connected nodes (hubs) are extremely rare in random graphs. The degree distribution of a random graph follows a Poisson distribution, which implies that most nodes have a degree close to the average degree. The probability function for a Poisson distribution is given by: ««««< HEAD

$$P(k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

=======

$$P(k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

»»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872 where λ is the average degree of the network.

Scale free graphs, on the other hand, exhibit a highly heterogeneous structure. These networks are grow over time, and new nodes preferentially attach to alread welll-connected nodes. This process leads to the emergence of hubs, which are nodes with a significantly higher degree than the average node. Consequently, scale-free networks are characterized by strong degree ineuqality. Scale-free networks follow a power-law degree distribution of the form: ««««< HEAD

$$P(k) \sim k^{-\gamma}$$

=======

$$P(k) \sim k^{-\gamma}$$

»»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872 where γ is a positive constant typically in the range of 2 to 3. This power-law behavior indicates that while most nodes have a low degree, a few nodes (hubs) have a very high degree. Below is a visual representation of the degree distribution for random and scale-free networks.

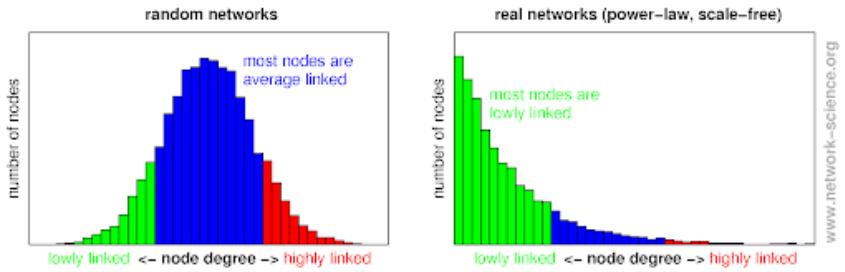


Figure 4: Comparison of random and scale-free networks and their corresponding degree distributions. Adapted from Science n.d.

«««< HEAD ======= »»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872

2.2.3 Structural Implications

The degree distribution of a network has significant implications for its structure and dynamics. In networks with a Poisson degree distribution, connectivity is relatively uniform, meaning most of the nodes have the the same amount of connections and there are rarely outliers. Therefore, the network does not depend heavily on individual nodes. In contrast, scale-free networks are dominated by hubs, which play a crucial role in maintaining overall connectivity. The structural difference affects various network properties. For instance scale-free networks are typically robust against random node failures, as most nodes are low-degree nodes. However, they are highly vulnerable to targeted attacks on hubs. Furthemore, the presence of hubs strongly influences centrality measures and also modularity. As a result, understanding the degree distribution is essential for intepreting community detection results and structural properties. Centrality measures and Modularity will be discussed in the following sections.

2.3 Centrality Measures

2.3.1 Degree centrality

Degree centrality measures a node's importance in a network by counting its total number of connections (degree). In an undirected network, the degree centrality of a node is simply the number of edges connected to it. It can be expressed mathematically as: «««< HEAD

$$C_D(v) = \deg(v)$$

where $\deg(v)$ is the degree of node v. In a directed network, degree centrality can be further divided into in-degree centrality and out-degree centrality. It can be expressed mathematically as:

$$C_{D_{in}}(v) = \deg_{in}(v)$$

$$C_{D_{out}}(v) = \deg_{out}(v)$$

=======

$$C_D(v) = \deg(v)$$

where $\deg(v)$ is the degree of node v. In a directed network, degree centrality can be further divided into in-degree centrality and out-degree centrality. It can be expressed mathematically as:

$$C_{D_{in}}(v) = \deg_{in}(v)$$

$$C_{D_{out}}(v) = \deg_{out}(v)$$

»»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872 where $\deg_{in}(v)$ is the number of incoming edges to node v and $\deg_{out}(v)$ is the number of outgoing edges from node v.

2.3.2 Betweenness centrality

Betweenness centrality measures the importance of a node by quantifying how often it lies on shortest paths between other pairs of nodes. More formally, it counts the proportion of shortest paths between all node pairs that pass through a given node. Mathematically, the betweenness centrality of a node v is defined as: «««< HEAD

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

=======

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

»»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872 where σ_{st} is the total number of shortest paths from node s to node t, and $\sigma_{st}(v)$ is the number of those paths that pass through node v. Nodes with a high betweeness centrality often act as bridges between different parts of the network, frequently connecting otherwise weakly connected communities. Ednge betweenness extends the same idea to edges instead of nodes. It measures how often an edge lies on the shortest paths between pairs of nodes. The betweenness centrality of an edge e is defined as: «««< HEAD

$$C_B(e) = \sum_{s \neq t} \frac{\sigma_{st}(e)}{\sigma_{st}}$$

=======

$$C_B(e) = \sum_{s \neq t} \frac{\sigma_{st}(e)}{\sigma_{st}}$$

»»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872 where σ_{st} is the total number of shortest paths from node s to node t, and $\sigma_{st}(e)$ is the number of those paths that pass through edge e. This characteristic is particularly important in community detection methods based on edge betweenness, such as the algorithm introduced by M. G. Newman M. E. J. (2002), which will be discussed in more detail in the section on community detection methods.

2.4 Modularity

The core idea behind modularity is, a network has a strong community structure if:

- there are many edges between nodes within the same community.
- there are few edges between nodes in different communities.

Therefore, modularity compares the observed structure to a random baseline. This means that modularity measures how much more densely connected nodes within the same community are than we would expect by chance. The modularity formula is given by: «««< HEAD

$$Q = \frac{1}{2m} \sum_{ij} (A_{ij} - P_{ij}) \delta(c_i, c_j)$$

=======

$$Q = \frac{1}{2m} \sum_{ij} (A_{ij} - P_{ij}) \delta(c_i, c_j)$$

»»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872 where m is the total number of edges in the network, A_{ij} is the adjacency matrix element representing the presence of an edge between nodes i and j, P_{ij} is the expected number of edges between nodes i and j in a random graph with the same degree distribution, and $\delta(c_i, c_j)$ is the Kronecker delta function that equals 1 if nodes i and j are in the same community (i.e., $c_i = c_j$) and 0 otherwise. The modularity value ranges theoretically between -1 and 1, although in practice it typically lies between 0 and 1. Higher modularity values indicate a stronger community structure, meaning that more edges fall within communities than would be expected in a comparable random network. A known limitation of modularity is the resolution limit problem, meaning that small communities may not be detected because they do not contribute enough to the overall modularity score.

2.5 Clustering Coefficient

2.5.1 Definition and interpretation

A third property that many networks have in common is clustering, or network transitivity, which is the property that two vertices that are both neighbors of the same third vertex have a heightened probability of also being neighhbors of one another. For example in the language of a protein protein interaction network, if two proteins interact with the same third protein, they are more likely to interact with each other as well. This effect is quantified by the clustering coefficient C, defined by: «««< HEAD

$$C = \frac{3 \times \text{number of triangles}}{\text{number of connected triples}}$$

=======

$$C = \frac{3 \times \text{number of triangles}}{\text{number of connected triples}}$$

»»»> 61f5ba4ae41d0db3608f6be7f4fbb2e75927a872 where a triangle is a set of three nodes that are mutually connected, and a connected triple is a set of three nodes where at least two of them are connected.

3 Similarity Measures for Community Comparison

3.1 Adjusted Rand Index (ARI)

The Adjusted Rand Index (ARI) is a metric that measures the similarity between two partitions of the same set of nodes. A partition refers to the complete division of nodes into clusters. THE ARI considers all pairs of nodes and evaluates whether each pair is assigned consistently to the same community or to different communities in both partitions.

The classical Rand Index (RI) evaluates the proportion of node pairs that are either placed in both partitions or in different partitions in both clusterings. However, since random clusterings can also exhibit non-zero agreement, the ARI corrects this value by adjustion for chance.

Formally, the ARI is defined as:

$$ARI = \frac{RI - E[RI]}{\max(RI) - E[RI]} \tag{1}$$

where RI is the Rand Index, $E[RI]$ is the expected Rand Index for random clusterings, and $\max(RI)$ is the maximum possible Rand Index. The ARI ranges from -1 to 1, where:

- 1 indicates perfect agreement between the two partitions,
- 0 indicates that the agreement is no better than random,
- Negative values indicate less agreement than expected by chance.

3.2 Adjusted Mutual Information (AMI)

The Adjusted Mutual Information (AMI) is another metric used to compare two partitions based on information theory. It measures the mutual information between two clusterings, quantifying how much knowing one partition helps to predict the cluster assignments of the other.

Mutual Information (MI) measures the amount of information shared between two partitions. However, like the RI, MI can be biased by random agreement. The AMI corrects for this by substracting the expected mutual information under a random model.

The AMI is defined as:

$$AMI = \frac{MI - E[MI]}{\max(MI) - E[MI]} \tag{2}$$

where MI is the Mutual Information, $E[MI]$ is the expected Mutual Information for random clusterings, and $\max(MI)$ is the maximum possible Mutual Information. The AMI also ranges from -1 to 1, with similar interpretations as the ARI:

- 1 indicates perfect agreement,
- 0 indicates agreement no better than random,
- Negative values indicate less agreement than expected by chance.

3.3 Comparison of ARI and AMI

Both ARI and AMI are widely used for evaluating the similarity between two partitions, however, they differ conceptually and computationally. The ARI is pair-counting based and focused on whether pairs of nodes are assigned consistently across partitions. It is relatively easy to intepret. However, it can be sensitive to imbalanced cluster sizes. On the other hand, AMI is inforamtion-theoretic and evaluates how much information is shared between clusterings. It is generally more robust to imbalanced cluster distribution and performs well when the number of clusters differs between partitions. The ARI is often preferred when interpretability and pairwise consistency are important, particulary in case of balanced cluster sizes.

The AMI is advantageous when cluster sizes vary significantly or when comparing partitions with different numbers of communities. Due to its information-theoretic nature, it provides a more distribution-sensitive comparison. Both Indexes are valuable tools for community comparison. Later in the results section, we will apply both ARI and AMI to compare different community detection results.

4 Community Detection Methods

In this section, we will explore the community detection methods that form the core of this study.

4.1 Edge Betweenness

The edge betweenness method, introduced by Girvan and Newman (2002), was made to sidestep the shortcomings of traditional community detection methods like the hierarchical clustering approach. The hierarchical clustering method focuses on trying to construct a measure that tells us which edges are most central to communities. Using these measures, it constructs communities by adding the strongest edges to an initially empty graph. However, this method has several drawbacks, such as the fact that it relies on a subjective choice of the similarity measure. Furthermore, because it tries to build communities by adding edges, early mistakes made by noise in the data cannot be corrected later on. The edge betweenness method, on the other hand, removed weak edges from the original graph, instead of adding edges. To find the weak edges of a graoh, Girvan and Newman generalized Freeman's betweenness centrality to edges. M. G. Newman M. E. J. (2002) states if a network contains communities or groups that are only loosely connected by fe integroup edges, then all shortest paths different communities must go along one of these few edges. Thus, the edges connecting communities will have high edge betweenness. By removing these edges, we seperate groups from one another and so reveal the underlying community structure of the graph. The algorithm they propose is simply stated as follows:

1. Compute the edge betweenness for all edges in the network.
2. Remove the edge with the highest edge betweenness.
3. Recalculate the edge betweenness for all edges affected by the removal.
4. Repeat from step 2 until no edges remain.

The iteration ends when no edges remain, so in praxis, we will have to decide when to stop the algorithm. This can be done by looking at the modularity of the network after each iteration and stopping when the modularity is maximized. This algorithm is computationally expensive, because we have to recalculate the edge betweenness after each iteration. Making the algorithm runs in worst-case time $O(m^2n)$, where m is the number of edges and n is the number of nodes in the graph. So it is tempting to calculate the betweenness of all edges only once and then remove them in order of decreasing betweenness. However, we find that this approach does not work well, because if two communities are connected by more than one edge, then there is no guarantee that all of the edges will have high betweenness. So by recalculating betweennesses after the removal of each edge, we can ensure that at least one of the remaining edges between two communities will always have high betweenness.

4.2 Louvain Method

The Louvain Algorithm introduced by Blondel et al. (2008) is a popular method for community detection in large networks. The core idea is to group nodes so that connections inside communities are denser, than expected at random. Louvain does this by greedy, local optmization of the modularity measure, combined with a hierarchical approach. The algorithm consists of two phases that are repeated iteratively until no further improvement in modularity is possible:

1. **Modularity Optimization:**

   - Initially, each node is assigned to its own community.
   - For each node i, we consider the moving it to the community of each neighbor.
   - The change in modularity ΔQ is calculated for each possible move, and the node is moved to the community that results in the highest increase in modularity. ΔQ is given by the formula:

     $$\Delta Q = Q(i \in C) - Q(i \in A)$$

     where $Q(i \in C)$ is the modularity of the network if node i is moved to community C, and $Q(i \in A)$ is the modularity if node i remains in its current community A. The formula for calculating modularity is as earlier given by:

     $$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \gamma \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

     where A_{ij} is the adjacency matrix of the graph, k_i and k_j are the degrees of nodes i and j, m is the total number of edges in the graph, c_i and c_j are the communities of nodes i and j, and $\delta(c_i, c_j)$ is the Kronecker delta function which is 1 if $c_i = c_j$ and 0 otherwise. γ is the resolution parameter that allows us to control the size of the communities detected. If γ is set to 1, we get the standard modularity. If γ is set to a value less than 1, we will get larger communities, and if γ is set to a value greater than 1, we will get smaller communities. We will play around with the resolution parameter later on in the sensitivity analysis section, to see how it affects the community structure of our networks.

2. **Community Aggregation:**

   - After no more individual node movements can increase modularity, we create a new network where each community is treated as a single node, also known as supernode.
   - Edges between supernodes are replaced by wweighted edges, where the weight of the edge between two supernodes is the sum of the weights of the edges between the nodes in the corresponding communities.
   - Edges within the same community are represented as self-loops

In the figure below, we can see an visualization of the Louvain method.

The hierarchical nature of the Louvain method allows it to capture community structure at multiple scales, and it is computationally efficient, with a time complexity of $O(n \log n)$ in practice, making it suitable for large networks. However there are several notable limitations to the Louvain method, firstly, it does not guarantee that the detected communities are internally connected. In practice, a community may consist of multiple disconnected components. Furthermore, the algorithm may converge to a local maximum of modularity, as it only ensures that no single node move can further increase modularity. These shortcomings motivated Traag, Waltman and van Eck (2019) to introduce the Leiden algorithm, which will be discussed in the following section.

4.3 Leiden Method

We here introduce the Leiden algorithm, which guarantees well connected communities. The algorithm is partly based on the previous introduced smart local moving algorithm, which is a faster version of the Louvain algorithm. The Leiden algorithm also takes advantage of the idea of speeding up the local moving of nodes and the idea of moving nodes to a random neighbour by only considering a subset of nodes that are likely to move.V.A. Traag (2019a) states that

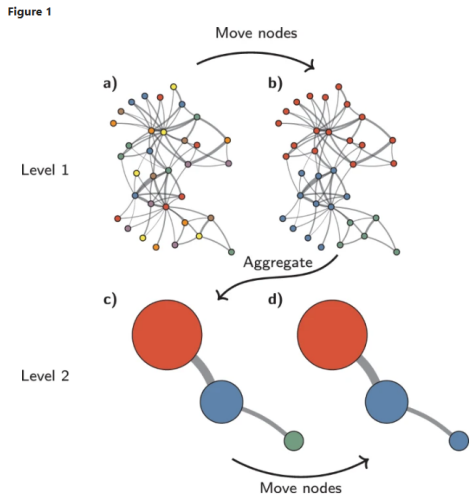


Figure 5: Visual representation of the Louvain method for community detection. Adapted fromV.A. Traag, L. Waltman, N.J. van Eck 2019.

they consider these ideas to represent the most promising the most promising directions in which Louvain algorithm can be improved, even though they recognise that other improvments have been suggested as well. The Leiden algorithm consists of three phases:

1. **Local Moving of Nodes:**
   - Similar to the Louvain method, each node is initially assigned to its own community.
   - For each node i, we consider moving it to the community of each neighbor, and calculate the change in modularity ΔQ for each possible move.
   - The node is moved to the community that results in the highest increase in modularity, but only if this move does not lead to disconnected communities.
2. **Refinement of Communities:**
   - After no more individual node movements can increase modularity, we refine the communities by splitting them into smaller subcommunities if it leads to an increase in modularity.
3. **Community Aggregation:**
   - Similar to the Louvain method, we create a new network where each community is treated as a single node, and edges between supernodes are replaced by weighted edges.

In the figure below, we can see an visualization of the Leiden method.

V.A. Traag (2019a) explains that in contrast to the Louvain algorithm, the Leiden algorithm introduces an additional refinement phase between the local moving phase and the aggregation phase. The purpose of this refinement step is to construct a partition that refines the partition obtained after the local moving phase. More precisely, if the partition resulting from the local moving phase is denoted by P, the refinement phase identifies a new partition P', where each community in P' is a subset of a community in P. Thus, communities in P may be subdivided into smaller, more internally cohesive subcommunities in P'.

Initially, the refined partition P' assigns each node to its own singleton community. Subsequently, nodes are merged within their original community in P, but only if such merges improve the quality function. Importantly, merging is restricted to nodes that belong to the

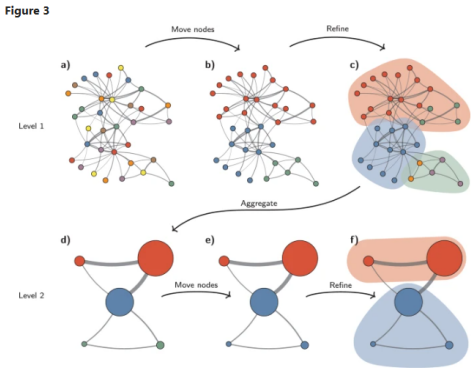


Figure 6: Visual representation of the Leiden method for community detection. Adapted fromV.A. Traag 2019c.

same community in P, ensuring that P' remains a refinement of P. After the refinement phase, communities in P' may consist of multiple nodes, yet they remain strictly contained within the original communities of P.

Compared to the Louvain algorithm, this refinement phase provides greater flexibility in exploring alternative partitions. Whereas Louvain performs greedy node movements without revisiting earlier decisions, Leiden systematically evaluates potential merges within communities. Merges are only accepted if they increase the quality function, thereby preventing deteriorations in partition quality. As a result, the algorithm is able to escape poorly connected or suboptimal community configurations.

Another important difference concerns the local moving phase itself. In the Louvain algorithm, nodes are considered in a fixed order, and once a node is moved, it is not reconsidered unless it is affected by subsequent movements. In contrast, the Leiden algorithm maintains a queue of nodes that require reevaluation. Whenever a node changes community, its neighbors are added to the queue to ensure that their potential moves are reconsidered. This dynamic updating procedure continues until no further improvements can be achieved. By revisiting affected nodes, Leiden avoids prematurely fixing suboptimal assignments and improves convergence toward higher-quality partitions.

4.3.1 Theoretical Guarantees of the Leiden Algorithm

The Leiden algorithm is executed iteratively, where the partition obtained in one iteration serves as the starting point for the next. This iterative structure allows the algorithm to progressively refine the community structure and provides several theoretical guarantees at different stages of convergence. Below, we give an intuitive overview of these guarantees. Formal definitions and mathematical proofs can be found in the supplementary material.

After each iteration of the Leiden algorithm, the following properties hold:

1. All communities are γ-separated.
2. All communities are γ-connected.

Here, γ denotes the resolution parameter of the quality function being optimized, such as modularity. The property of γ-separation implies that no pair of communities can be merged without decreasing the quality function. This property is also satisfied by the Louvain algorithm.

The property of γ-connectivity is stronger than ordinary connectivity. It guarantees that communities are internally well connected in a manner consistent with the resolution parameter. In contrast, the Louvain algorithm does not guarantee connectivity and therefore does not ensure γ-connectivity.

An iteration in which the partition remains unchanged is referred to as a *stable iteration*. After a stable iteration of the Leiden algorithm, two additional guarantees hold:

1. All nodes are locally optimally assigned.
2. All communities are subpartition γ-dense.

Local optimality of nodes means that no individual node can be moved to another community in a way that improves the quality function. This property is also guaranteed after a stable iteration of the Louvain algorithm.

Subpartition γ-density, however, is not guaranteed by Louvain. A community is subpartition γ-dense if it cannot be decomposed into internally separable parts without reducing the quality function. In other words, each community consists of well-connected components that themselves satisfy the same density condition recursively. This property ensures structural cohesion beyond simple node-level optimality.

Unlike the Louvain algorithm, the Leiden algorithm may still improve the partition even after a stable iteration. When iterated until full convergence, the Leiden algorithm yields a partition satisfying two stronger guarantees:

1. All communities are uniformly γ-dense.

2. All communities are subset optimal.

Uniform γ-density means that no subset of a community can be separated from it without decreasing the quality function. Regardless of how a community is divided into two parts, these parts remain sufficiently well connected. This implies that the overall partition is close to optimal with respect to the quality function.

Subset optimality represents the strongest guarantee of the Leiden algorithm. It states that every subset of every community is locally optimally assigned; that is, no subset of nodes can be moved to another community in a way that improves the objective function. Subset optimality implies uniform γ-density and all previously mentioned properties.

In summary, while the Louvain algorithm guarantees only local node-level optimality and γ-separation, the Leiden algorithm provides substantially stronger guarantees, ensuring well-connected, structurally cohesive, and near-optimal communities.

Here is an Overview of the guarantees provided by the Louvain algorithm and the Leiden algorithm. Due to its stronger theoretical guarantees and improved partition quality, the Leiden

		Louvain	Leiden
Each iteration	γ-separation	✓	✓
	γ-connectivity		✓
Stable iteration	Node optimality	✓	✓
	Subpartition γ-density		✓
Asymptotic	Uniform γ-density		✓
	Subset optimality		✓

Figure 7: Visual representation of the guarantees of the Leiden algorithm. Adapted from V.A. Traag 2019b.

algorithm is widely regarded as a state-of-the-art method for community detection in large-scale networks.

References

European Bioinformatics Institute (EMBL-EBI) (2024). *Network analysis of protein interaction data – Types of biological networks*. Accessed: 27 February 2026. URL: `https://www.ebi.ac.uk/training/online/courses/network-analysis-of-protein-interaction-data-an-introduction/types-of-biological-networks/`.

Hevey, David (2018). "Network analysis: a brief overview and tutorial". In: *PMC*.

Newman, M. E. J. (2006). "Modularity and community structure in networks". In: *PNAS*.

Newman M. E. J., M. Girvan (2002). "Community structure in social and biological networks". In: *PNAS*.

ResearchGate (2020). *Different types of graphs and their corresponding adjacency matrix representations*. Accessed: 27 February 2026. URL: `https://www.researchgate.net/figure/Different-types-of-graphs-and-their-corresponding-adjacency-matrix-representations-The_fig1_347300725`.

Science, Network (n.d.). *Node Degree Distribution:Random networks versus scale-free power-law networks*. URL: `http://www.network-science.org/powerlaw_scalefree_node_degree_distribution.html`.

V.A. Traag L. Waltman, N.J. van Eck (2019a). "From Louvain to Leiden: guaranteeing well-connected communities". In: *Scientific Reports*. URL: https://www.nature.com/articles/s41598-019-41695-z/tables/1.

— (2019b). *Overview of the guarantees provided by the Louvain algorithm and the Leiden algorithm*. URL: https://www.nature.com/articles/s41598-019-41695-z/tables/1.

— (2019c). *Visual representation of the Leiden method for community detection*. URL: https://www.nature.com/articles/s41598-019-41695-z.

V.A. Traag, L. Waltman, N.J. van Eck (2019). *Visual representation of the Louvain method for community detection*. URL: https://www.nature.com/articles/s41598-019-41695-z.